

Implement SSH, firewall, automation, and SELinux on a Linux server.

Task: Submit a comprehensive report on server security configurations.

TASKS

1.Enable and configure SSH on your Linux server so that only key-based authentication is allowed and password authentication is disabled.

Configure SSH (Key-Based Authentication Only)

Step 1: Install OpenSSH

```
dnf install openssh-server -y
```

Step 2: Start SSH

```
systemctl enable sshd  
systemctl start sshd
```

Step 3: Verify

```
systemctl status sshd
```

```
[root@localhost ~]# systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: ena
   Active: active (running) since Fri 2026-07-17 11:09:48 IST; 1h 49min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
    Main PID: 7324 (sshd)
      Tasks: 1 (limit: 4273)
     Memory: 120.0K (peak: 5.9M)
        CPU: 99ms
       CGroup: /system.slice/sshd.service
              └─7324 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jul 17 11:09:48 localhost.localdomain systemd[1]: Starting OpenSSH server daemo>
Jul 17 11:09:48 localhost.localdomain sshd[7324]: Server listening on 0.0.0.0 p>
Jul 17 11:09:48 localhost.localdomain sshd[7324]: Server listening on :: port 2>
Jul 17 11:09:48 localhost.localdomain systemd[1]: Started OpenSSH server daemon.
Jul 17 11:11:16 localhost.localdomain sshd-session[7375]: Accepted password for>
Jul 17 11:11:17 localhost.localdomain sshd-session[7375]: pam_unix(sshd:session>
lines 1-18/18 (END)
```

Step 4:

Generate SSH Key (Normal User)

```
su - du
```

```
ssh-keygen
```

Press **Enter** for all questions.

Step 5: Copy Public Key

```
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

Permissions:

```
chmod 700 ~/.ssh
```

```
chmod 600 ~/.ssh/authorized_keys
```

Step 6: Disable Password Login

Return to root

```
exit
```

Edit SSH configuration

```
vi /etc/ssh/sshd_config
```

Find

```
PasswordAuthentication yes
```

Change to

```
PasswordAuthentication no
```

Save

```
Esc
```

```
:wq
```

Restart SSH

```
systemctl restart sshd
```

Verify

```
grep PasswordAuthentication /etc/ssh/sshd_config
```

```
[root@localhost ~]# grep PasswordAuthentication /etc/ssh/sshd_config
#PasswordAuthentication no
# PasswordAuthentication. Depending on your PAM configuration,
# PAM authentication, then enable this but set PasswordAuthentication
[root@localhost ~]#
```

2. Set
up a
basic

firewall using UFW or firewalld to allow only SSH (port 22) and HTTP (port 80) traffic, and block all other incoming connections.

Hint: After applying rules, use a port scanning tool like nmap from another machine to verify only the allowed ports are open.

Configure Firewall

CentOS uses **firewalld**.

Check status

systemctl status firewalld

Start firewall

systemctl enable firewalld

systemctl start firewalld

Allow SSH

firewall-cmd --permanent --add-service=ssh

Allow HTTP

firewall-cmd --permanent --add-service=http

Reload

firewall-cmd --reload

Verify

firewall-cmd --list-services

```
success
[root@localhost ~]# firewall-cmd --list-services
cockpit dhcpv6-client http ssh
[root@localhost ~]#
```

Verify with Nmap

dnf install nmap -y

nmap localhost

```
[root@localhost ~]# nmap localhost
Starting Nmap 7.92 ( https://nmap.org ) at 2026-07-17 13:06 IST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000023s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
631/tcp   open  ipp
3306/tcp  open  mysql

Nmap done: 1 IP address (1 host up) scanned in 0.57 seconds
[root@localhost ~]#
```

3. Write a shell script `automate_backup.sh` that creates a backup of your `/home` directory and saves it with a timestamp in `/backups`.

Backup Script

Create backup directory

`mkdir -p /backups`

Create script

`vi /root/automate_backup.sh`

Paste

`#!/bin/bash`

`DATE=$(date +%F_%H-%M-%S)`

`tar -czf /backups/home_backup_$(date).tar.gz /home`

`echo "Backup completed at $(date)"`

Save

Esc

:wq

Make executable

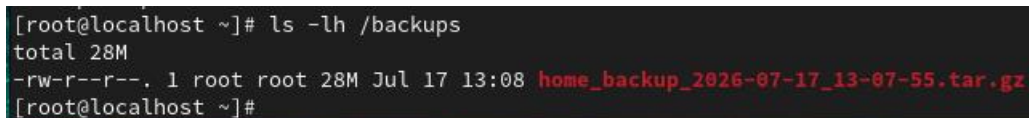
chmod +x /root/automate_backup.sh

Run

/root/automate_backup.sh

Verify

ls -lh /backups



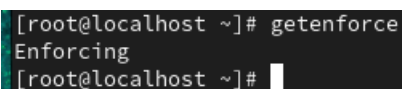
```
[root@localhost ~]# ls -lh /backups
total 28M
-rw-r--r--. 1 root root 28M Jul 17 13:08 home_backup_2026-07-17_13-07-55.tar.gz
[root@localhost ~]#
```

4. Check the current status of SELinux on your server, switch it to enforcing mode if it is not already, and verify the change using appropriate commands.
Hint: Use the commands **getenforce** and **setenforce**.

SELinux

Check mode

getenforce



```
[root@localhost ~]# getenforce
Enforcing
[root@localhost ~]#
```

5. Use ChatGPT or Copilot to generate a checklist of at least 5 best practices for securing a Linux server, and compare it with your own server configuration to identify any missing steps.

Generate a Security Checklist

AI Prompt

Generate a checklist of at least 5 best practices for securing a Linux server.

AI-Generated Checklist

Best Practice	Description
Keep the operating system updated	Install security updates regularly using dnf update.
Use SSH key authentication	Use SSH keys instead of passwords for remote login.
Disable SSH password login	Set PasswordAuthentication no in /etc/ssh/sshd_config.
Configure the firewall	Allow only required ports such as SSH (22) and HTTP (80).
Keep SELinux in Enforcing mode	Protect the system by enforcing SELinux security policies.
Perform regular backups	Schedule automatic backups using cron jobs.

```
[root@localhost ~]# vi /root/security_checklist.txt
[root@localhost ~]# cat /root/security_checklist.txt
Linux Server Security Checklist

SSH Installed : Yes
SSH Key Authentication : Configured
Password Authentication Disabled : Yes
Firewall Enabled : Yes
SSH Port 22 Allowed : Yes
HTTP Port 80 Allowed : Yes
SELinux Enforcing : Yes
Automatic Backup Script : Created
[root@localhost ~]#
```